Chapter 9
The Blockchain

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content


Introduction


Structure of a Block


Block Header


Block Identifiers


The Genesis Block


Linking Blocks in the Blockchain

Content



Merkle Trees



Merkle Trees and Simplified Payment Verification (SPV)

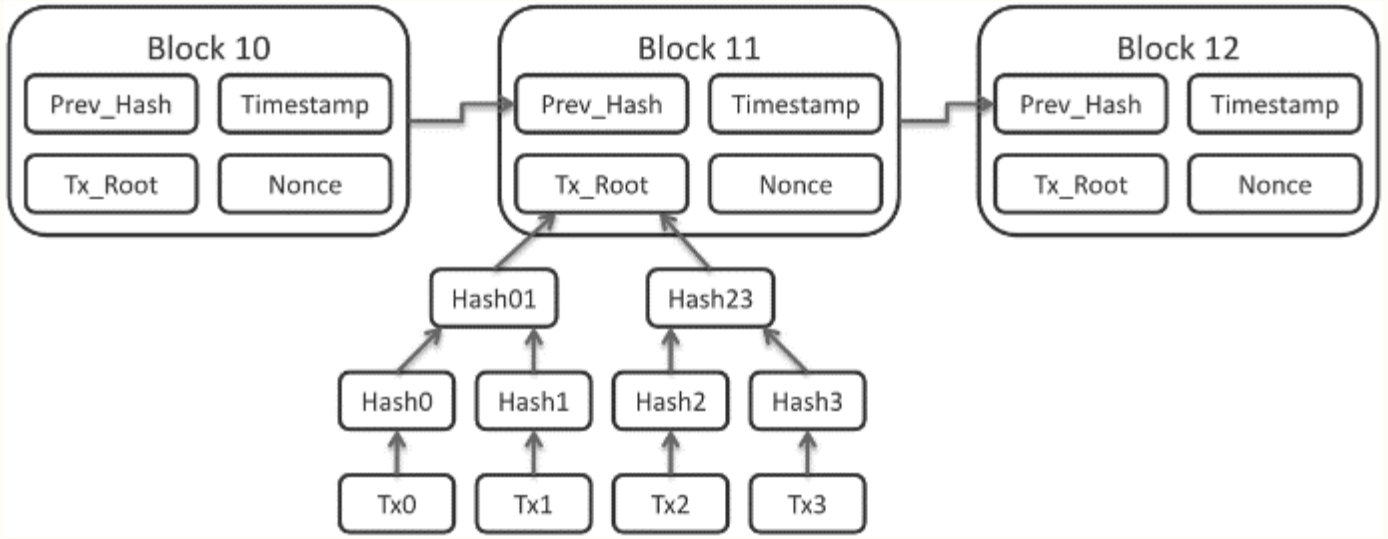


Bitcoin’s Test Blockchains



Using Test Blockchains for Development

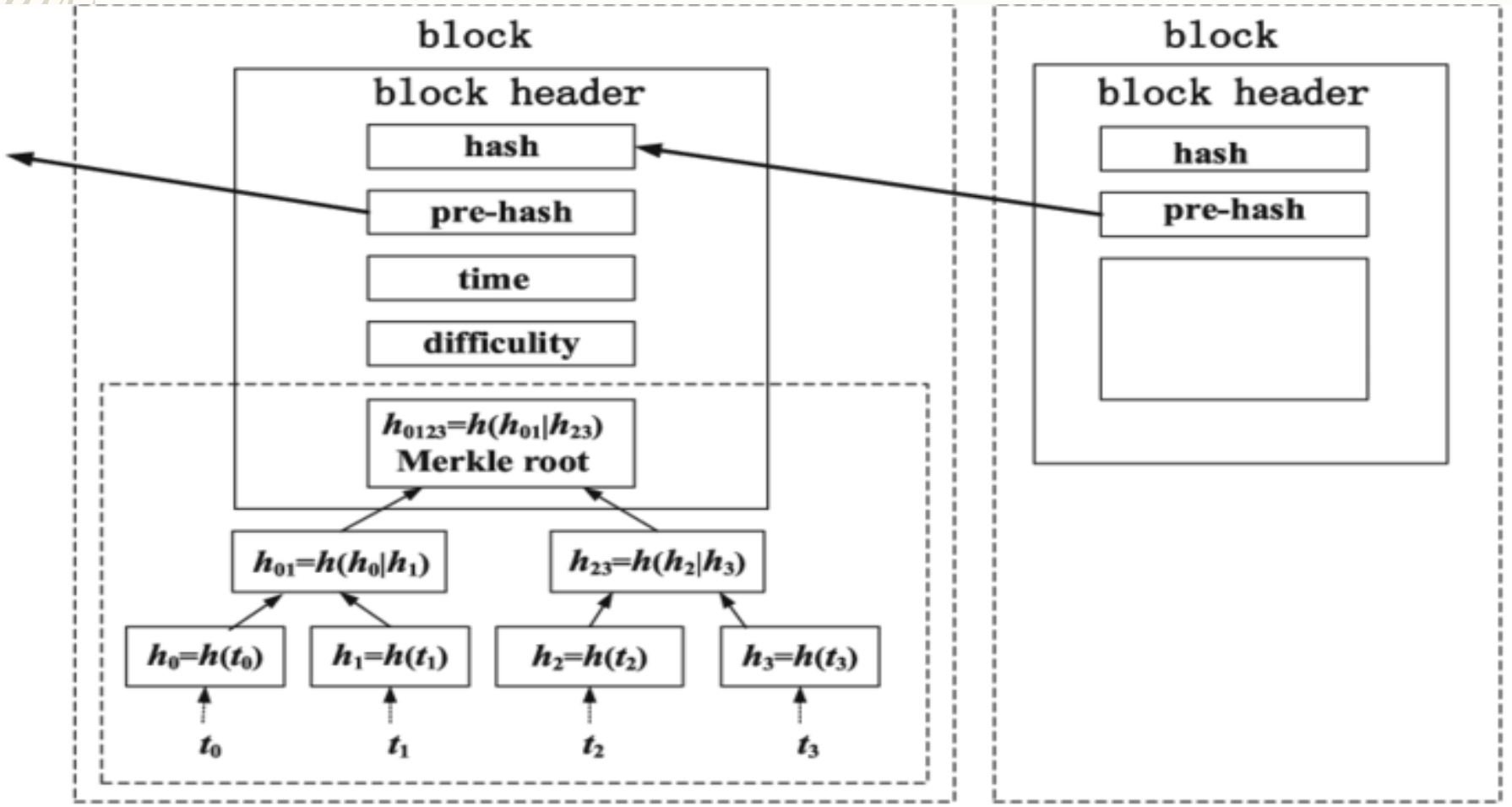
Introduction



Introduction

Blockchain data structure	• Ordered • Back-linked • List of transaction blocks
Storage	• Can be stored as a flat file • Or in a simple database
Height	• The distance from the first block
Top	• The most recently added block

Blockchain Data Structure



https://www.researchgate.net/publication/327616607_Using_Blockchain_for_Data_Auditing_in_Cloud_Storage_4th_International_Conference_ICCCS_2018_Haikou_China_June_8-10_2018_Revised_Selected_Papers_Part_III/figures?lo=1&utm_source=google&utm_medium=organic

Introduction

Block Header

Keeps data

Block fields

Fields

Identifier

Version number

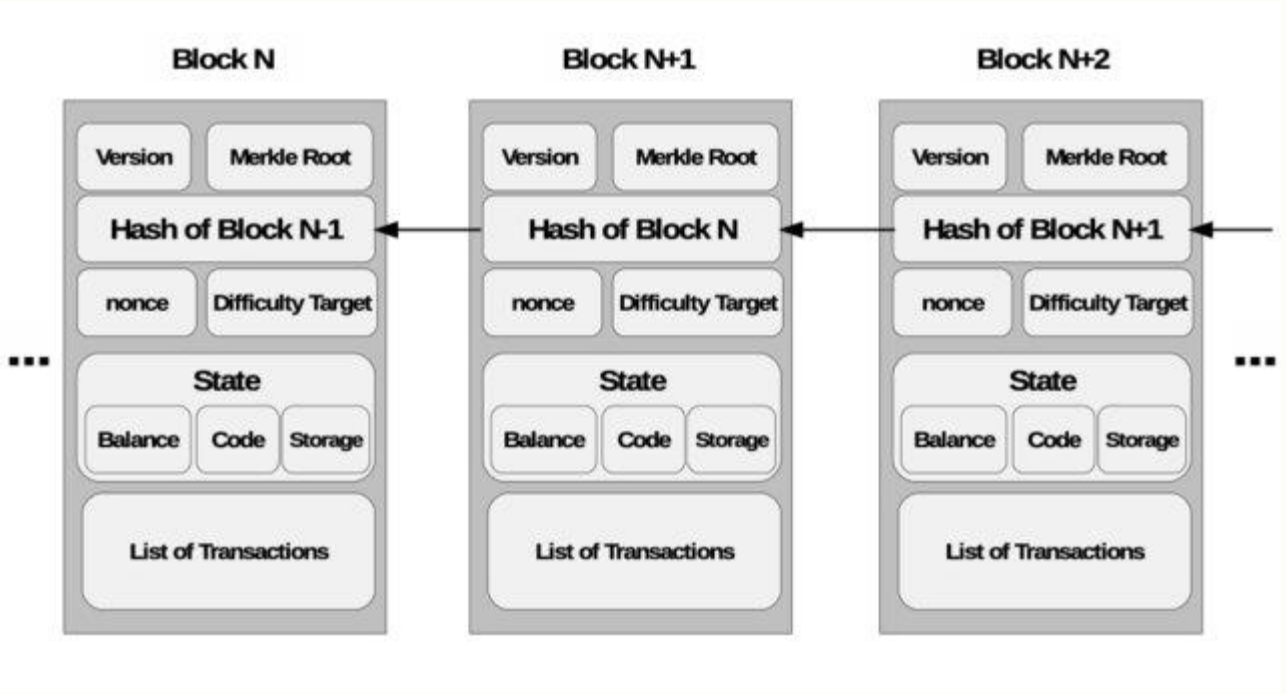
Previous block hash

Merkle root

Time stamp

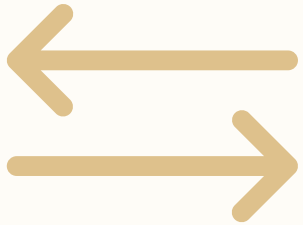
The difficulty target

Nonce



https://www.researchgate.net/publication/321017113_IoT_Security_Review_Blockchain_Solutions_and_Open_Challenges/figures?lo=1

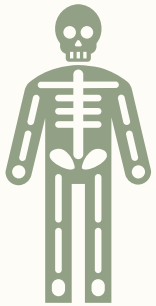
Introduction



Chain

The sequence of blocks

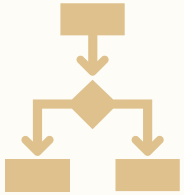
Going back all the way



Genesis Block

the first block ever created

Fork Problem



Fork

A temporary situation



Multiple children

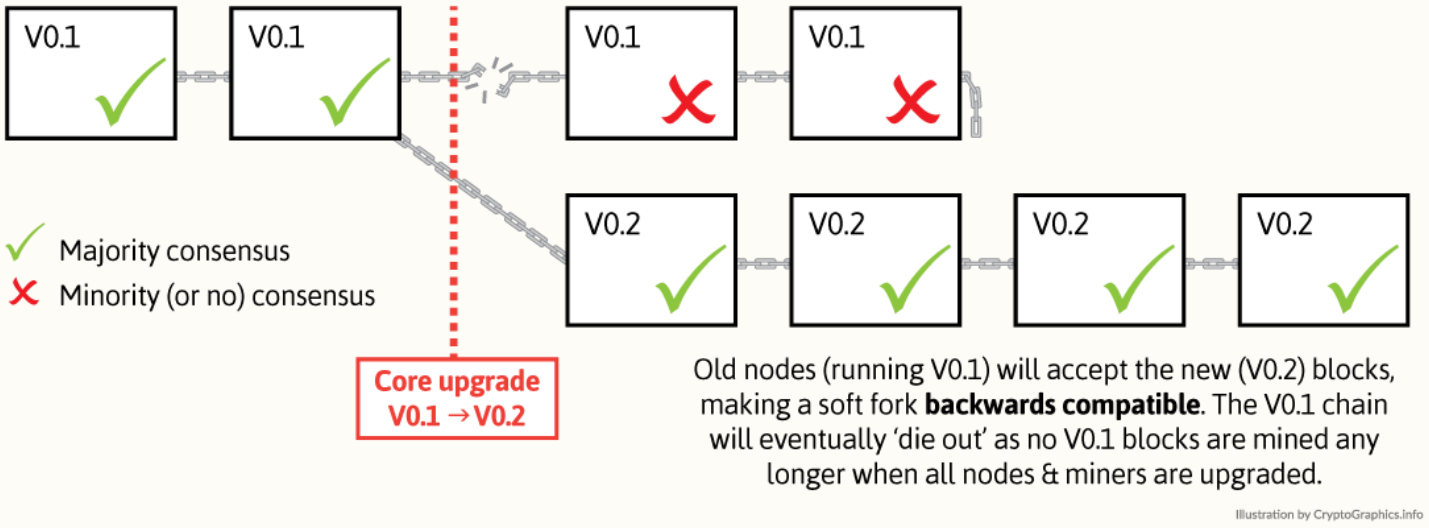
references to the same parent

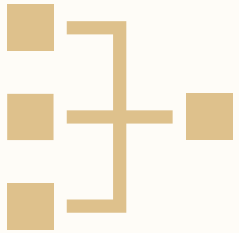


Reason

miners discover different blocks simultanously

Resolving Fork Problem





Resolution

Only one children becomes part of the block chain

https://cryptographics.info/cryptographics/blockchain/hard-soft-forks/

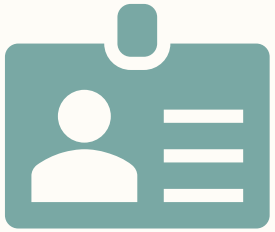
Immutability



Chaining

The «previous block hash»

affects affects the current block’s hash



Parent <- Child

change to parent identity

invalidates child’s own identity

Immutability



Fasten the Chain

change a block

requires recalculation of

all subsequent blocks



Recalculation

require enormous computation

and energy consumption

Immutability



100 Blocks

Stability

the Coinbase transaction can be spent



Coinbase transaction

the transaction

containing newly mined bitcoin



1000 blocks

the blockchain is settled

for all practical purposes.



Structure of a Block

Block as a Data Structure

Structure of a Block



Block

is a container



Container data structure

aggregates transactions

for inclusion in the public ledger

Structure of a Block

Block

Magic number

Block size

Header

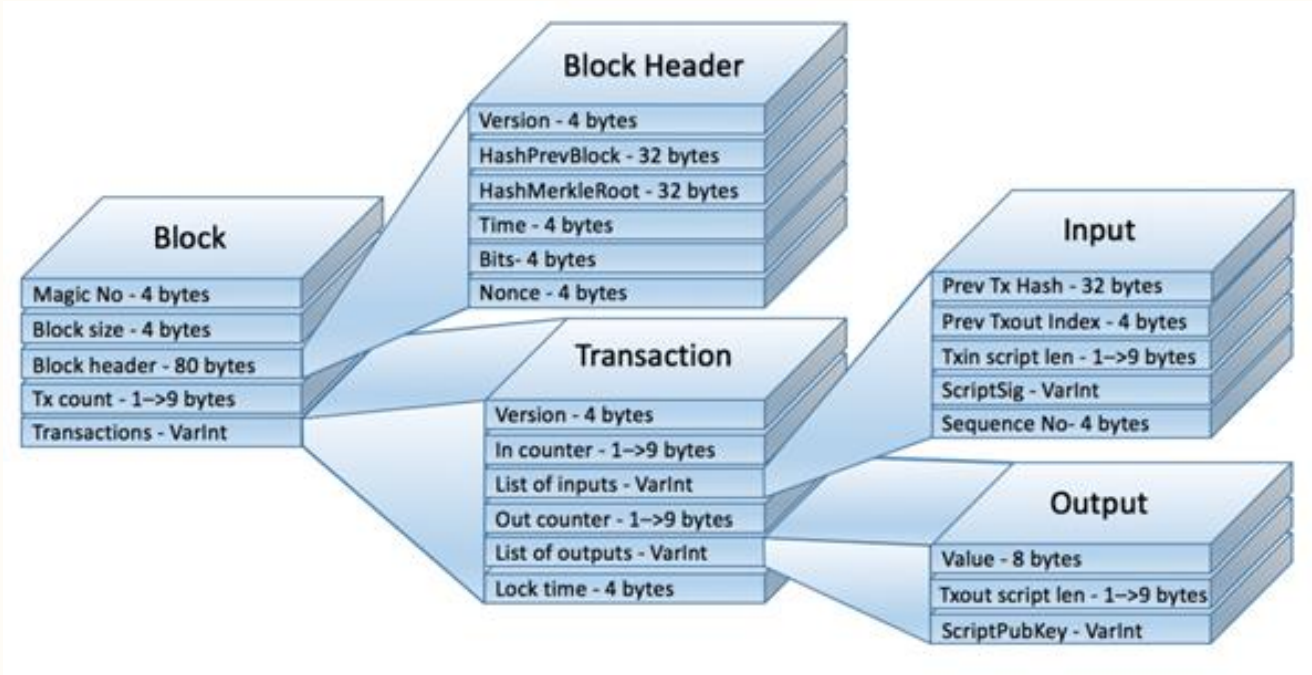
Transaction count

Transactions

header

80 bytes

containing metadata



Structure of a Block



250 Bytes

average size

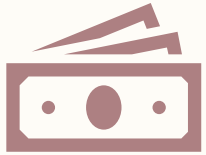
of a transaction



500 Transactions

of transactions

in a block



1000 times larger

Size of block

comparing the block header

Table 9-1. The structure of a block

Size	Field	Description
4 bytes	Block Size	The size of the block, in bytes, following this field
80 bytes	Block Header	Several fields form the block header
1–9 bytes (VarInt)	Transaction Counter	How many transactions follow
Variable	Transactions	The transactions recorded in this block



Block Header

Fields of Header

Block Header



Header content

three sets of block metadata.



Set-1

previous block hash



Set-2

the difficulty

timestamp

Nonce



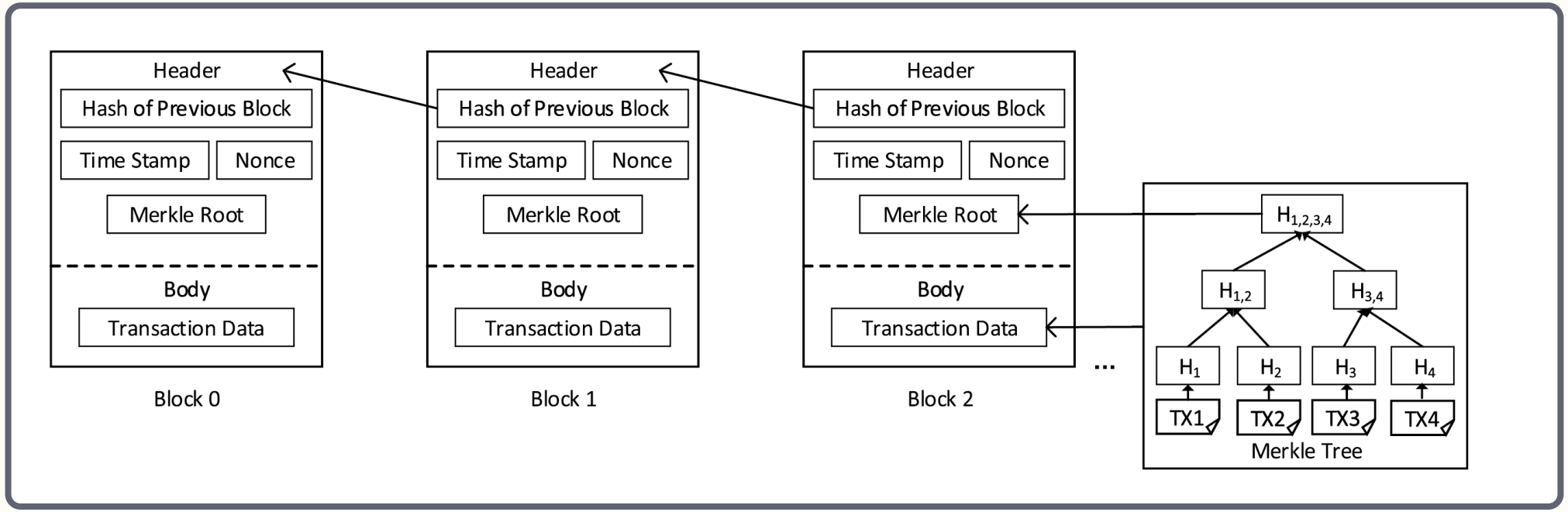
Set-3

Merkle tree



merkle tree

a data structure efficiently summarize the transactions



The structure of a Blockchain

Taken from Blockchain for Dynamic Spectrum Management

Table 9-2. The structure of the block header

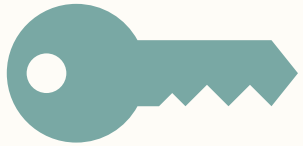
Size	Field	Description
4 bytes	Version	A version number to track software/protocol upgrades
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash of the root of the merkle tree of this block's transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)
4 bytes	Difficulty Target	The Proof-of-Work algorithm difficulty target for this block
4 bytes	Nonce	A counter used for the Proof-of-Work algorithm



Block Identifiers

Block Header Hash and Block Height

Block Header Hash



The primary identifier

Block’s cryptographic hash



Digital fingerprint

Made by hashing the header twice

Through SHA256: 32 Byte

Block Header Hash



Hash of Block header

only fields of the block header

is used to compute the identifier



Block 0

Header hash

000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

Block Hash: No Part of Block's Data Structure



Computet Value

Block hash

not in block's data structure



Confirmation

Computed by each node

when the block is transmitted



Not stored

on a node's persistence

as part of the blockchain

Block Hash



Separate storage

might be stored

in a separate database table

as part of the block’s metadata



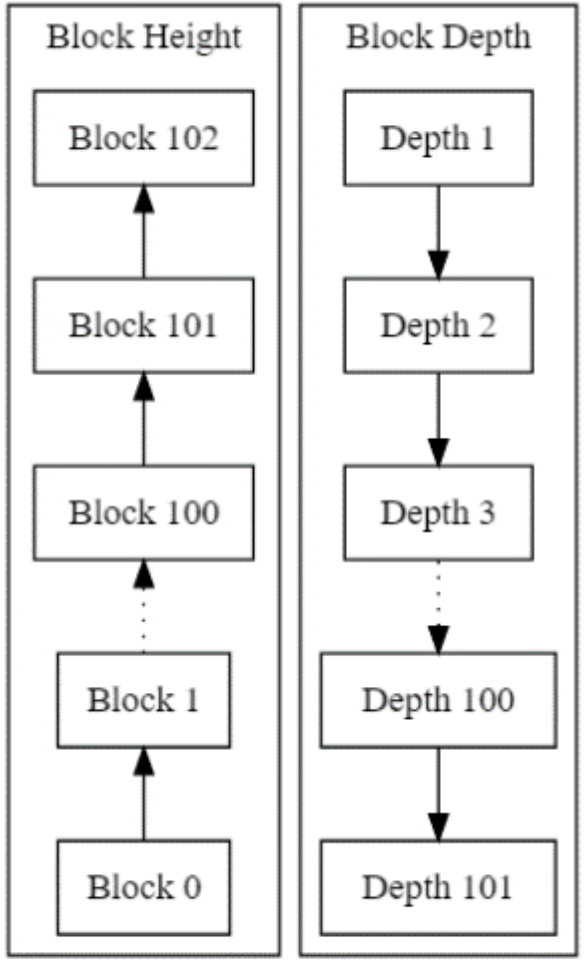
Reason

Faster Access:

facilitate indexing and

faster retrieval of blocks from disk

Identifying a Block by its Position



Block Height Compared To Block Depth

Block Height

of blocks since January 2009

height 0

The first block ever created is

at block height 0 (zero)

one position higher

subsequent block

added “on top” of the first block

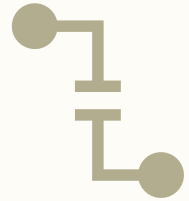
Properties of Block Height



Not unique

Two or more blocks

might have the same block height



Reason: Forking

competing for the same position

in the blockchain



The Genesis Block

Hidden Message

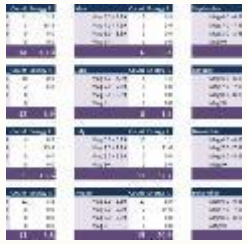
Genesis Block



Genesis Block

1st block in the blockchain

Block 0



2009

The creation time

03.01.2009 21:15



The common ancestor

the common ancestor of

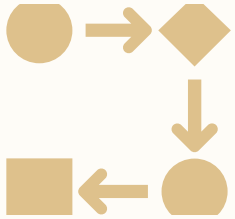
all the blocks

Genesis Block



Traceble

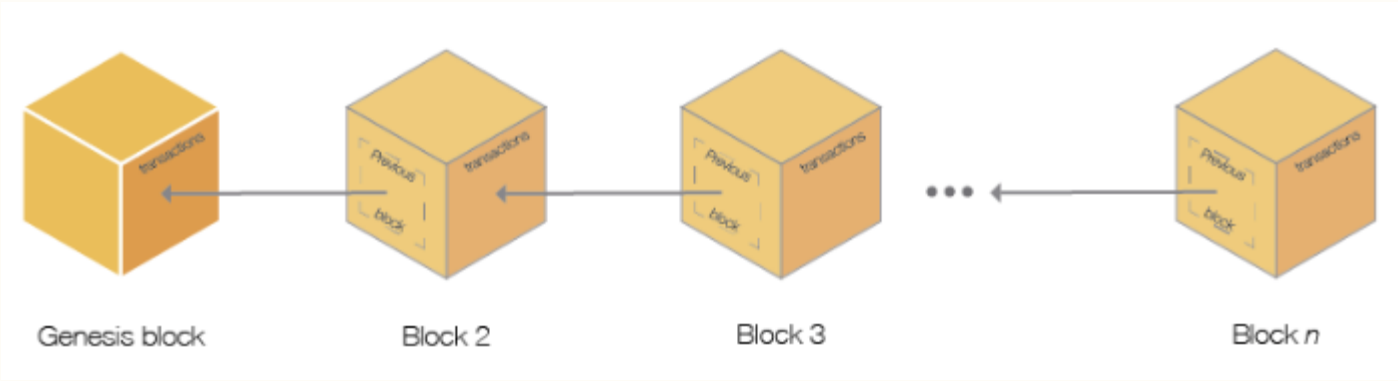
from any block



Arrive genesis block

start at any block

and follow the chain backward



Genesis Block



Fixed

Genesis block is

cannot be altered



Static Block

statically encoded



SW Data

included within

the bitcoin client software



Every node knows

hash

structure

creation time

the single transaction

Genesis Block



Genesis block

is a secure “root”

to build a trusted blockchain.



See in bitcoin core client

Github link



block explorer

website

such as blockchain.info

Search link

Hidden Message



Proof of the date

message

to offer proof of the earliest date



The Times

the headline of

the British newspaper *The Times*



Message

“The Times 03/Jan/2009 Chancellor on brink of second bailout for banks.”

Hidden Message

The Times 03/Jan/2009

Chancellor on brink of second bailout for banks

Hash 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

Summary View details

Age	11 years ago
Date	January 3, 2009 9:15 PM UTC+03:00
Height	0
Confirmations	631712
Size	0.285 kB



THE TIMES

Saturday January 3 2009 timesonline.co.uk No 69523 £1.50

Eat Out from £5

More than 900 great restaurants, including four Gordon Ramsay favourites from £15

Start collecting tokens today Pullout Inside

Israel prepares to send tanks and troops into Gaza

Israel allowed foreigners to flee the Gaza Strip as it prepared for a ground offensive. At least 430 Palestinians were killed in a week of airstrikes News, page 3

Chancellor on brink of second bailout for banks

Billions may be needed as lending squeeze tightens

Michael Sheen Frost, Nixon and me Magazine

Working mums So that's how she does it Body&Soul

Detox in style The best spas on the planet Travel

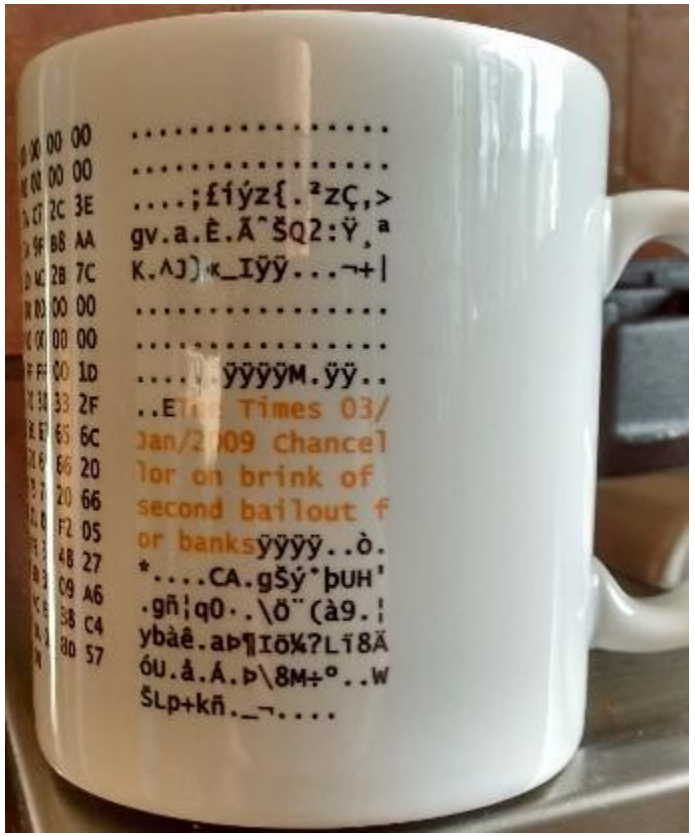
Salmon Rushdie I Won't Marry Again Pages 22, 23

Giant Killing? Guide to the FA Cup Third Round Sport

```
00000000  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000020  00 00 00 00 3B A3 ED FD  7A 7B 12 B2 7A C7 2C 3E  ....;£íýz{.²zÇ,>
00000030  67 76 8F 61 7F C8 1B C3  88 8A 51 32 3A 9F B8 AA  gv.a.È.Ã^ŠQ2:Ÿ¸ª
00000040  4B 1E 5E 4A 29 AB 5F 49  FF FF 00 1D 1D AC 2B 7C  K.^J)«_Iÿÿ...¬+|
00000050  01 01 00 00 00 01 00 00  00 00 00 00 00 00 00 00  ................
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000070  00 00 00 00 00 00 FF FF  FF FF 4D 04 FF FF 00 1D  ......ÿÿÿÿM.ÿÿ..
00000080  01 04 45 54 68 65 20 54  69 6D 65 73 20 30 33 2F  ..EThe Times 03/
00000090  4A 61 6E 2F 32 30 30 39  20 43 68 61 6E 63 65 6C  Jan/2009 Chancel
000000A0  6C 6F 72 20 6F 6E 20 62  72 69 6E 6B 20 6F 66 20  lor on brink of
000000B0  73 65 63 6F 6E 64 20 62  61 69 6C 6F 75 74 20 66  second bailout f
000000C0  6F 72 20 62 61 6E 6B 73  FF FF FF FF 01 00 F2 05  or banksÿÿÿÿ..ò.
000000D0  2A 01 00 00 00 43 41 04  67 8A FD B0 FE 55 48 27  *....CA.gŠý°þUH'
000000E0  19 67 F1 A6 71 30 B7 10  5C D6 A8 28 E0 39 09 A6  .gñ¦q0·.\Ö¨(à9.¦
000000F0  79 62 E0 EA 1F 61 DE B6  49 F6 BC 3F 4C EF 38 C4  ybàê.aÞ¶Iö¼?Lï8Ä
00000100  F3 55 04 E5 1E C1 12 DE  5C 38 4D F7 BA 0B 8D 57  óU.å.Á.Þ\8M÷°..W
00000110  8A 4C 70 2B 6B F1 1D 5F  AC 00 00 00 00           ŠLp+kñ._¬....
```






S. NAKAMOTO



Linking Blocks in the Blockchain

Making Blockchain Stable

Linking Incoming Nodes



Full node

maintain

a local copy of blockchain



Full chain

all blocks

starting at the genesis block



Linking

incoming blocks

valiadet and linked

to the existing blockchain



“previous block hash”

to establishing a link

examine the incoming block header

Figure 9-1. Blocks linked in a chain by reference to the previous block header hash

The last block the node knows about is block 277,314, with a block header hash of:
000000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249

previousblockhash field
000000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249

The node adds this new block to the end of the chain, making the blockchain longer with a new height of 277,315

new block received from the network:


```
{
    "size" : 43560,
    "version" : 2,
    "previousblockhash" :
        "000000000000000027e7ba6fe7bad39faf3b5a83daed765f05f7d1b71a1632249",
    "merkleroot" :
        "5e049f4030e0ab2debb92378f53c0a6e09548aea083f3ab25e1d94ea1155e29d",
    "time" : 1388185038,
    "difficulty" : 1180923195.25802612,
    "nonce" : 4215469401,
    "tx" : [
        "257e7497fb8bc68421eb2c7b699dbab234831600e7352f0d9e6522c7cf3f6c77",

 #[... many more transactions omitted ...]

        "05cfd38f6ae6aa83674cc99e4d75a1458c165b7ab84725eda41d018a09176634"
    ]
}
```



Merkle Trees

Construction

Merkle Trees



Merkle tree

Binary hash tree

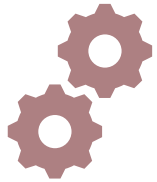
A data structure



transactions

summary of

all the transactions

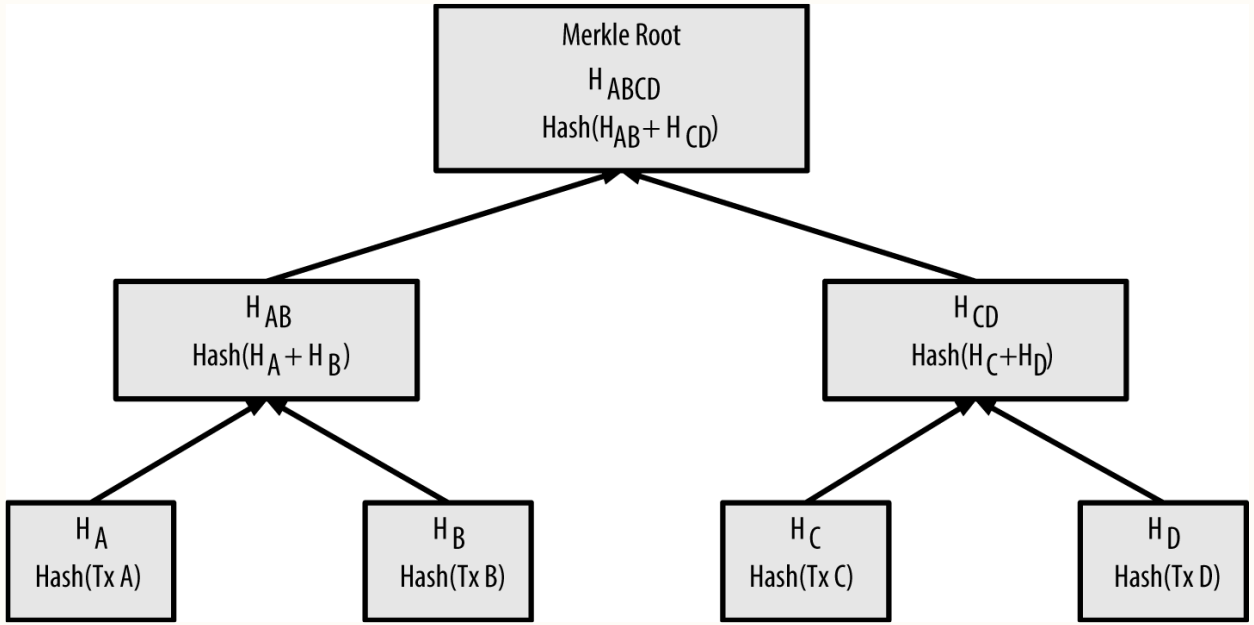


Efficiency

Transaction verification

summarizing

Merkle Trees



Merkle trees

are Binary trees

contain cryptographic hashes

digital fingerprint

of the entire set of transactions

Merkle Trees



Verify

if a transaction

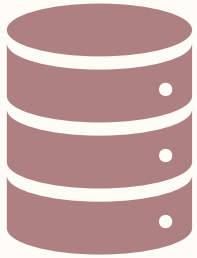
is included in a block



Efficient

very efficient in verification

Construction of Merkle Tree



Recursive hashing

A merkle tree is constructed

by recursively hashing pairs of nodes

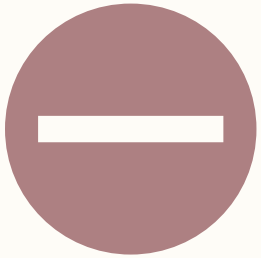


Stop condition

until there is only one hash

called the *root*, or *merkle root*.

Construction of Merkle Tree



Double-SHA256

SHA256 applied twice

also known as double-SHA256



Bottom-up construction

Constructed bottom-up

Starting with leaves

Construction Example



4 Transactions

start with four transactions, A, B, C, and D

which form the *leaves* of the merkle tree



Transaction Hash

The transactions are

not stored in the merkle tree

Construction Example



Hashing

the resulting hash is stored in each leaf node



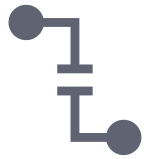
as

H_A

H_B

H_C

H_D



H_A =

SHA256(SHA256(Transaction A))

Construction Example



Parent

Summary of

Consecutive pairs of leaf nodes



Parent's hash

concatenating the two hashes

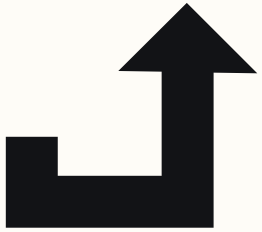
hashing them together



H_{AB}

= SHA256(SHA256(HA + HB))

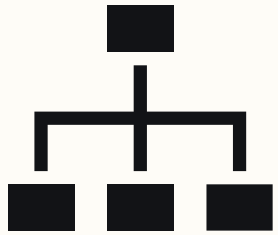
Construction Example



Continue

The process continues

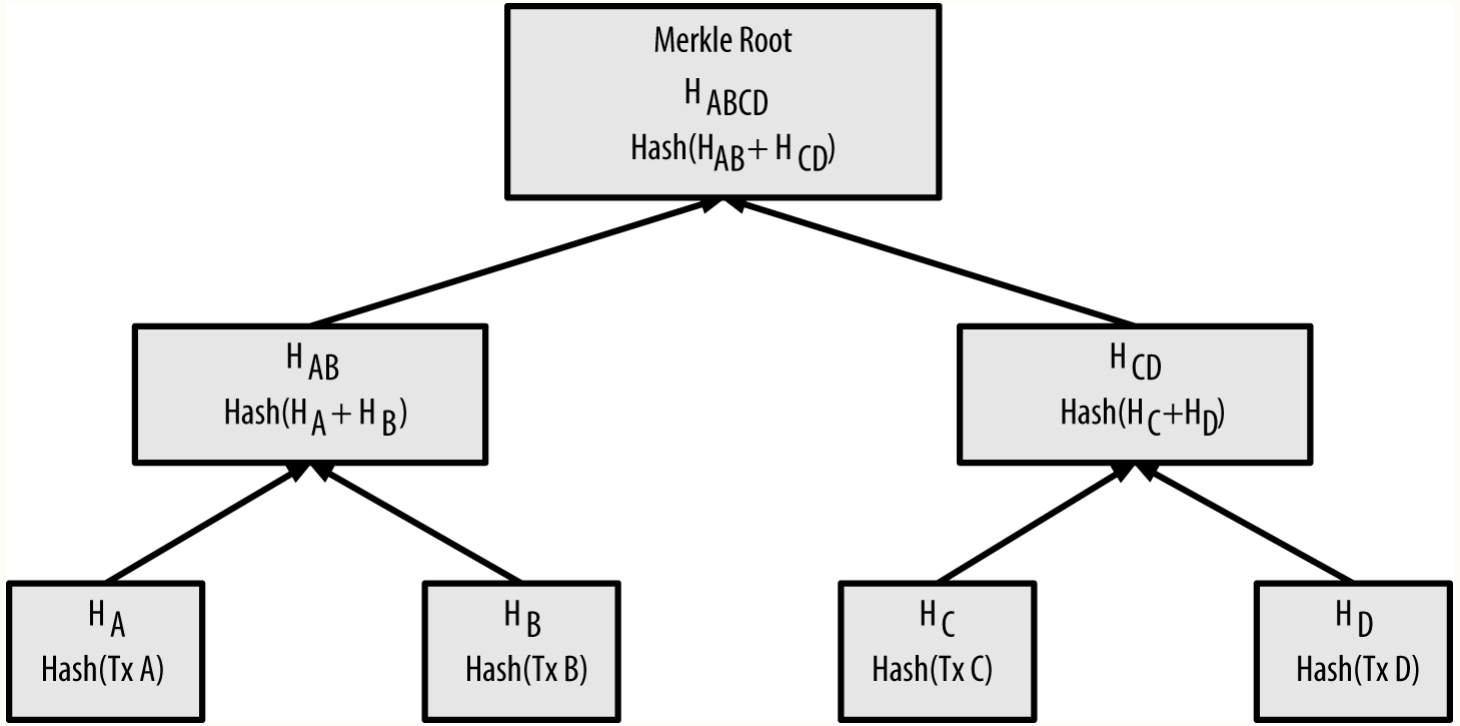
until there is only one node at the top



merkle root

the node known as the merkle root

Construction Example





TELUSKO

Merkle Tree | Merkle Root | Blockchain

https://www.youtube.com/watch?v=fB41w3JcR7U

Table 9-3. Merkle tree efficiency

Number of transactions	Approx. size of block	Path size (hashes)	Path size (bytes)
16 transactions	4 kilobytes	4 hashes	128 bytes
512 transactions	128 kilobytes	9 hashes	288 bytes
2048 transactions	512 kilobytes	11 hashes	352 bytes
65,535 transactions	16 megabytes	16 hashes	512 bytes



Merkle Trees and Simplified Payment Verification (SPV)

Efficiency Gain

Merkle Trees and Simplified Payment Verification (SPV)



Usage by SPV Nodes

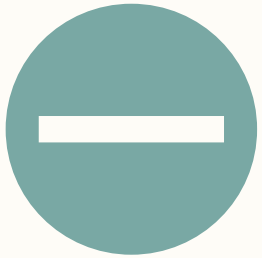
Merkle trees are used
extensively by SPV nodes.



Just Block Headers

SPV nodes don’t have all transactions
do not download full blocks
just block headers.

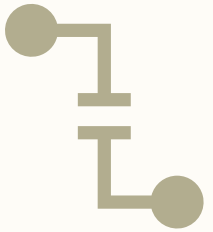
Merkle Trees and Simplified Payment Verification (SPV)



Verify with merkle path

they use an authentication path, or merkle path

to verify that a transaction is included in a block,



No need to the blocks

without having to download

all the transactions in the block

Work of SPV with Merkle Trees



merkle block message

When a peer sees a transaction

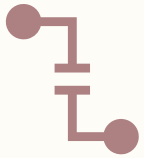
that matches the bloom filter



it will send

that block

using a merkleblock message.



Content

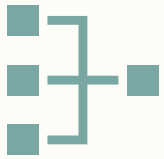
the block header

a merkle path

links the transaction

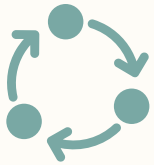
to the merkle root

Work of SPV with Merkle Trees



SPV Node

can use this merkle path



to connect

the transaction to the block

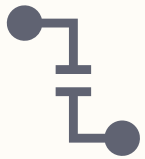


to verify

the transaction is

included in the block.

Gain of Efficiency



Data

the SPV node will have received

less than a kilobyte of data



Message content

the block header

merkle path



Comparing

amount of data is

more than a thousand times less

than a full block: about 1 megabyte

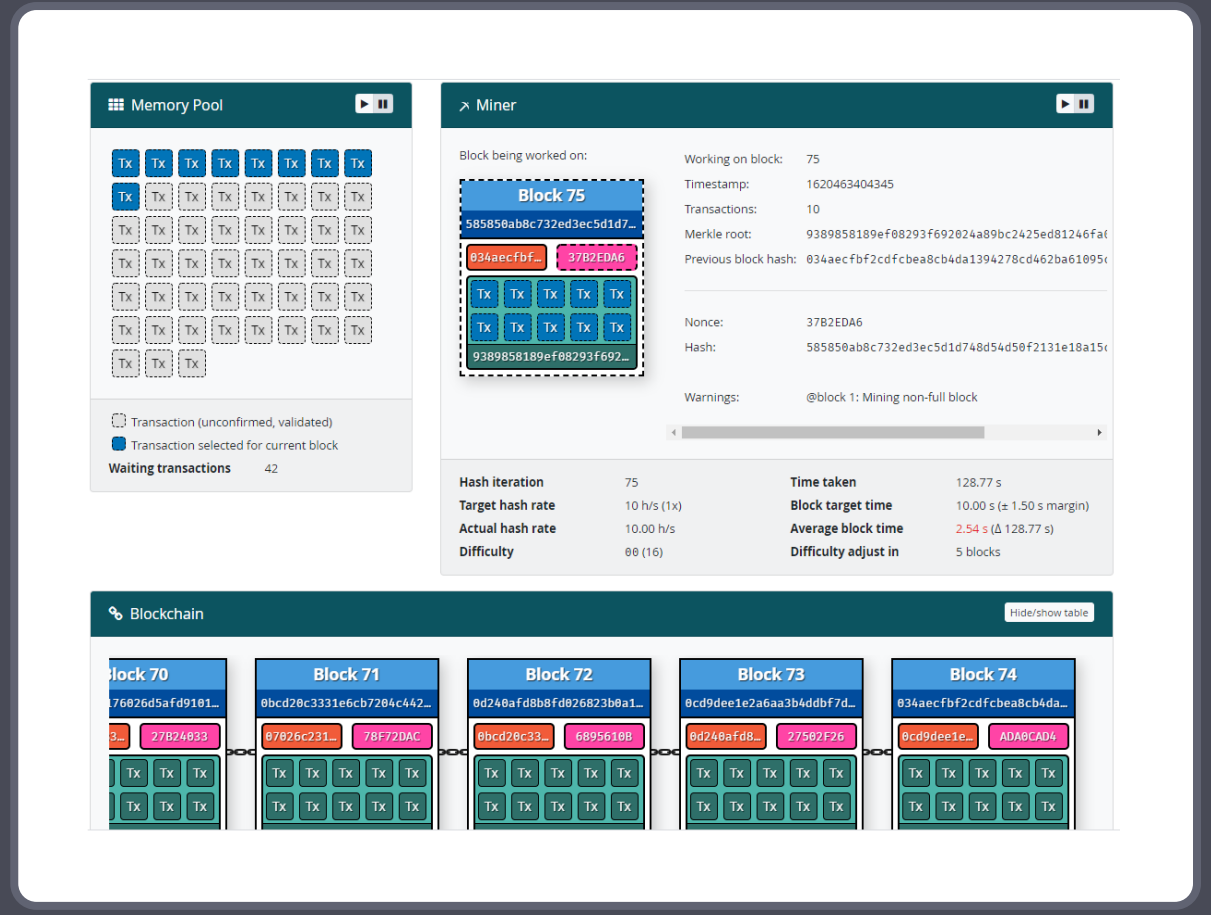


Blockchain Simulator

Blockchain Sim - v0.2 (beta)

JavaScript Blockchain Simulator

https://code.hnldesign.nl/blockchain/





Bitcoin's Test Blockchains

Testnet—Bitcoin's Testing Playground

Segnet—The Segregated Witness Testnet

Regtest—The Local Blockchain

More than One Bitcoin Blockchain



How many bitcoin blockchains?

there is more than one bitcoin blockchain.



Mainnet

The “main” bitcoin blockchain,

the one created by Satoshi Nakamoto on January 3rd, 2009,

the one with the genesis block is called *mainnet*

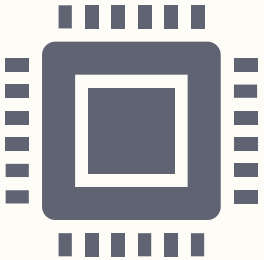


Other bitcoin blockchains

There are other bitcoin blockchains used for testing purposes

at this time *testnet*, *segnet*, and *regtest*.

Testnet—Bitcoin's Testing Playground



Testnet

is the name of the test blockchain, network, and currency

that is used for testing purposes.



Fully featured

The testnet is a fully featured live P2P network,

with wallets, test bitcoins (testnet coins), mining, and all the other features of mainnet.

Testnet—Bitcoin's Testing Playground



differences

Two differences



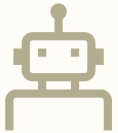
testnet coins are meant

to be worthless



mining difficulty

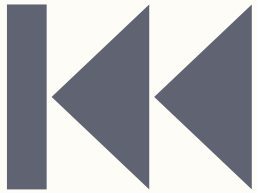
should be low enough



Easy mining

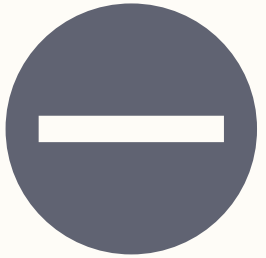
anyone can mine testnet relatively easily

testnet3



restarting

the testnet has to be scrapped
and restarted from a new genesis block,
resetting the difficulty



testnet3

The current testnet is called *testnet3*
restarted in February 2011 to reset the difficulty
from the previous testnet

Segnet—The Segregated Witness Testnet

Segregated Witness

- In 2016, a special-purpose testnet was launched to aid
- in development and testing of Segregated Witness

Added to testnet3

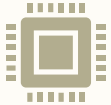
- Since segwit was added to testnet3
- it is no longer necessary to use segnet
- for testing of segwit features

Regtest—The Local Blockchain



regtest

stands for “Regression Testing,”



Usage

allows you to create a local blockchain

for testing purposes



Closed system

the regtest blockchains are intended to be run

as closed systems for local testing.



Own genesis block

You launch a regtest blockchain from scratch,

creating a local genesis block.



Options

You may add other nodes to the network,

or run it with a single node only to test the Bitcoin Core software.



Using Test Blockchains for Development

Development Pipeline

Using the Test Blockchains



Development pipeline

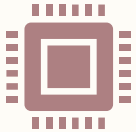
You can use the test blockchains

to establish a development pipeline.



Regtest after development

Test your code locally on a regtest as you develop it.



Switch to testnet

to try it on a public network switch to testnet

to expose your code to a more dynamic environment

with more diversity of code and applications.



Switch to mainnet

When code works as expected,

switch to mainnet to deploy it in production.

Discussions

Your opinion

What do you think on this chapter?

Challenging

What is the most interesting point?

Clarification

What are the unclarified points?

Critical issue

Select and explain a critical issu in this chapter

